ЛАБОРАТОРНА РОБОТА №11
Функціональне програмування:
Вирази-генератори та спискові вирази. Ітератори. Генератори.

Мета: дослідити основні техніки функціонального програмування засобами мови програмування Python.

Загальні вимоги до виконання лабораторної роботи:
- Написати консольний додаток, що реалізує завдання згідно умови ***мовою Python.***
- Вхідні дані подаються користувачем на консоль.
- Результати відображаються користувачу у консольному режимі.

Індивідуальні завдання

Завдання 1.1. ***Парні числа та спискові вирази***

Студенту необхідно підібрати функцію, яка використовує цикл for, щоб створити новий список значень. Ваше завдання – перепишіть цю функцію, **використовуючи генератор списків замість циклу for**.

Приклад функції, яка відфільтровує парні числа зі списку, використовуючи цикл for, та створюючи новий список значень:

```
def filter_even_numbers(nums: list[int]) -> list[int]:
    result = []
    for num in nums:
        if num % 2 == 0:
            result.append(num)
    return result
print(filter_even_numbers([1, 2, 3, 4, 5]))
```

Результат: [2, 4, 8, 12]

Підказки до реалізації:
1) Використайте вираз на кшталт [<вираз> for <елемент> in <масив> if <умова>] як основу.
2) Не використовуйте цикли for чи метод .append() у реалізації.

Завдання 1.2. ***Ітератор зворотного відліку***

Реалізуйте власний клас-ітератор Countdown, який генерує числа у зворотному порядку — починаючи з вказаного цілого числа start і закінчуючи нулем (включно).

Ваш клас має реалізовувати протокол ітератора: методи _ _iter_ _() та _ _next_ _().

Template:

```
class Countdown:
    def __init__(self, start: int):
        # Write your code here
        pass

if __name__ == "__main__":
    for n in Countdown(5):
        print(n)
```

Принципи конструювання ПЗ

Підказки щодо реалізації:

1) Зберігайте поточне значення у змінній екземпляра та зменшуйте його при кожній ітерації.
2) Кидайте виняток StopIteration, коли значення стане менше нуля.
3) Можна передбачити захист від від'ємного початкового значення — ітерація одразу завершується.

Приклад:

```
c = Countdown(5)
    for number in c:
        print(number)              #[5 4 3 2 1 0]

print(list(Countdown(3)))          #[3, 2, 1, 0]
print(list(Countdown(0)))          #[0]
print(list(Countdown(-3)))         #[]
```

<u>Завдання 1.3.</u> ***Генератор діапазону з плаваючою крапкою***

Реалізуйте генераторну функцію float_range(), яка працює подібно до вбудованої функції range(), але підтримує дійсні числа (з плаваючою крапкою) та повертає значення ліниво (за допомогою yield). Генератор повинен створювати послідовність чисел від start до stop (не включаючи stop) з кроком step.

Функція повинна коректно працювати у випадках:

- додатного та від'ємного кроку;
- дробових кроків (наприклад, 0.1);
- граничних випадків, як-от start>=stop при додатному кроці.

Сигнатура функції:

def float_range(start: float, stop: float, step: float) -> Iterator[float]:

Template:

```
from typing import Iterator
def float_range(start: float, stop: float, step: float) ->
Iterator[float]:
            # Write your code here

print(list(float_range(1.0, 2.0, 0.3)))
```

Підказки до реалізації:

1) Використовуйте цикл while та yield.
2) Залежно від знаку кроку:
   - якщо step>0, зупиняйтесь, коли current>=stop;
   - якщо step<0, зупиняйтесь, коли current<=stop.
3) Уникайте проблем із точністю дійсних чисел — заокруглюйте результат, наприклад, через round(...).
4) Якщо step==0, піднімайте помилку ValueError.

Приклади:

- Приклад 1: Крок додатній, від 1.0 до 2.0 (не включно) [1.0, 1.3, 1.6, 1.9]
  print(list(float_range(1.0, 2.0, 0.3)))

- Приклад 2: Крок від'ємний, від 5.0 до 3.0 (не включно) [5.0, 4.5, 4.0, 3.5]
  print(list(float_range(5.0, 3.0, -0.5)))
- Приклад 3: Малий крок 0.1, беремо тільки перші 3 елементи. Генерує нескінченну кількість чисел, але ми зупиняємося на 3-х перших
  print(list(float_range(0.0, 1.0, 0.1))[:3])
- Приклад 4: Початок і кінець однакові – генератор нічого не повертає
  print(list(float_range(0.0, 0.0, 1.0))) #[]
- Приклад 5: Крок від'ємний, але «start<stop» – умова ніколи не виконається. Генератор нічого не повертає
  print(list(float_range(1.0, 2.0, -1.0))) #[]

<u>Завдання 1.4.</u> ***Рекурсивний обхід дерева***

Реалізуй генераторну функцію walk_tree, яка рекурсивно проходить дерево у вигляді вкладених словників (dict) та повертає всі ключі у порядку глибини (depth-first traversal).

Використовуйте конструкцію yield from для делегування ітерації вкладеним генераторам.

Сигнатура функції:

def walk_tree(data: dict) -> Iterator[str]:

Template:

```
from typing import Iterator
def walk_tree(data: dict) -> Iterator[str]:
    pass
     # Write your code here

tree = {"a": {"b": {"c": 1}}, "d": 2}
print(list(walk_tree(tree)))
```

Підказки до реалізації:

1) Перевіряйте, чи значення є словником (dict), і якщо так – рекурсивно обходьте його.
2) Для рекурсивного виклику використовуйте yield from.
3) Не враховуйте значення – лише ключі.

Приклади:

```
data = {
...     "a": {
...         "b": {
...             "c": 1
...         }
...     },
...     "d": 2
... }
print(list(walk_tree(data)))          #['a', 'b', 'c', 'd']

data = {
...     "x": {"y": {"z": {}}},
...     "m": {"n": 42}
... }
print(list(walk_tree(data)))          #['x', 'y', 'z', 'm', 'n']
```